

Generative Models for Image Denoising & Restoration

A Timeline-Based Literature Review — Foundation Models & Generative AI Course

Reference material for the "Small Generative Model for Image Denoising" Mini Competition

1. Why This Matters For You

Your task explicitly asks for a **generative model**. That changes the reference set: instead of just picking any restoration network, you should be looking at architectures that are trained the way generative models are trained — **GANs, VAEs, U-Net-based conditional generators, and Diffusion Models** (including Diffusion+Transformer hybrids). A plain feed-forward CNN (like a bare U-Net regression model) is still a very reasonable and useful building block — U-Net in particular is the *backbone* used inside almost every generative model below — but the "generative" framing usually means the model doesn't just predict one fixed clean image; it learns the underlying **distribution** of clean images and samples/reconstructs from it. This matters because it affects how well the model generalizes to unfamiliar damage: a generative model has an internal notion of "what a realistic clean image looks like," so it can push a restored output back toward realism even for corruption types it wasn't explicitly trained on.

Below is a **timeline-ordered** walk through the key papers, GitHub repos, and Hugging Face projects, organized by which generative architecture family they belong to: **GAN** → **Diffusion (DDPM)** → **Diffusion + Transformer / Latent Diffusion** → **All-in-One Blind Generative Restoration**.

2. Timeline Overview (At a Glance)

Year	Paper / Repo	Architecture Family	One-line Problem Solved
2015	U-Net (Ronneberger et al.)	CNN Encoder-Decoder (backbone)	Precise pixel-level prediction with limited data, via skip connections
2015	Sohl-Dickstein et al. — <i>Deep Unsupervised Learning using Nonequilibrium Thermodynamics</i>	Diffusion (theory)	First formal proposal of diffusion-based generative modeling
2017	DnCNN (Zhang et al.)	CNN (discriminative, not generative — included as the pre-generative baseline)	Learn to predict noise residual instead of the clean image directly
2017	Pix2Pix (Isola et al.)	Conditional GAN	General-purpose image-to-image translation, later reused for denoising/deblurring
2019	Song & Ermon — <i>Score-Based Generative Modeling</i>	Diffusion (score-matching theory)	Alternative mathematical foundation that DDPM later builds on
2020	DDPM (Ho, Jain, Abbeel)	Diffusion + U-Net	Makes diffusion models practically trainable and high quality for the first time
2022	DDRM (Kawar et al.)		

Year	Paper / Repo	Architecture Family	One-line Problem Solved
		Diffusion (pretrained, used at inference)	Reuse one pretrained generative diffusion model to solve <i>any</i> linear inverse problem (denoise, deblur, super-resolve, inpaint) without retraining
2022	SR3 (Saharia et al.)	Diffusion + U-Net (conditional)	Super-resolution via iterative refinement using a conditional DDPM
2022	Latent Diffusion Models / Stable Diffusion (Rombach et al.)	VAE + Diffusion (in latent space)	Makes diffusion generation dramatically cheaper by working in a compressed latent space instead of pixel space
2022	Restormer (Zamir et al.)	Efficient Transformer (U-Net-shaped)	Makes Transformer-based restoration computationally feasible at high resolution
2022	NAFNet (Chen et al.)	CNN (activation-free)	Strips out complexity to show a very cheap architecture can still match SOTA
2022	AirNet (Li et al.)	CNN + Contrastive Encoder	First blind all-in-one restoration model for <i>unknown</i> corruption
2023	DiffPIR (Zhu et al.)	Diffusion (plug-and-play, pretrained)	Uses a pretrained generative diffusion model as a "denoiser prior" inside a classic optimization loop
2023	PromptIR (Potlapalli et al.)	Transformer + learnable Prompts	Cheap, blind adaptation to different degradation types via small prompt tensors
2024	AdaIR (Cui et al.)	Transformer + Frequency modulation	Uses frequency-domain cues to generalize better to degradation types not seen in training
2024	ABAIIR (Serrano-Lozano et al.)	CNN/Transformer + LoRA adapters	Explicitly targets generalization to unseen degradations, with cheap adapter-based specialization
2024	GenDeg (Rajagopalan et al.)	Diffusion (for <i>training-data</i> generation)	Uses a diffusion model to synthesize a much more diverse set of corrupted training pairs, instead of relying on a small fixed list of hand-coded corruptions
2025	Defusion / Visual-Instructed Degradation Diffusion (Luo et al., CVPR 2025)	Diffusion + U-Net, visually-instructed	Unifies 8 different restoration tasks in one diffusion model, guided by "visual instructions" rather than text

3. GAN Era (2017 – 2019): Restoration as Image-to-Image Translation

3.1 Pix2Pix — "Image-to-Image Translation with Conditional Adversarial Networks" (Isola et al., CVPR 2017)

Problem it solved: How do you train a network to transform one type of image into another (e.g., sketches → photos, or in our case, corrupted → clean) when a simple pixel-difference loss (L1/L2) tends to produce blurry, "average" outputs?

Architecture, simply explained: A **conditional GAN**. A U-Net-shaped **generator** takes the corrupted image and outputs a restored image. A separate **discriminator** network looks at (input, output) pairs and tries to tell whether the output is the real clean image or something the generator faked. The generator is trained to fool the discriminator *while also* staying close to the true clean image (L1 loss), which pushes it toward sharp, realistic textures rather than blurry averages.

Novelty: Before this, most restoration networks used only pixel-wise losses, which tend to smear out fine detail. Pix2Pix's adversarial loss taught the network to care about **perceptual realism**, not just pixel accuracy — foreshadowing the DISTS-style perceptual metric your own competition scores you on.

Why it's relevant to you: Many later GAN-based denoisers (e.g., GAN-based real-noise modeling, R2C-GAN for X-ray restoration) are direct descendants of the Pix2Pix recipe: U-Net generator + adversarial loss + reconstruction loss. If you want a lightweight generative model rather than diffusion, a Pix2Pix-style conditional GAN with a small U-Net generator is a very implementable option.

3.2 GAN-Based Blind Denoising / Noise Modeling (multiple works, 2018-2019, e.g., Chen et al.; GAN2GAN)

Problem it solved: Real-world noise doesn't follow the clean, textbook Gaussian noise formula most synthetic training pairs use. These works use a GAN to **learn what real noise looks like** directly from data, then use that learned noise model to generate more realistic training pairs, or to denoise directly.

Architecture: A generator learns to produce noise samples matching the statistics of real noisy images (i.e., the GAN's "fake data" is noise, not clean images), and a discriminator distinguishes real vs. generated noise. This learned noise model is then used to make synthetic corrupted/clean training pairs far more realistic.

Why it's relevant to you: This is directly usable in your Section 3.1 (you generate your own corrupted/clean pairs). Instead of only using textbook Gaussian/Poisson noise functions, a small GAN-based noise model — even a simple one — trained on a handful of real noisy photos can make your synthetic training pairs more representative of real-world degradations, potentially helping generalization to the undisclosed held-out set.

4. Diffusion Era (2020 onward): Generative Denoising as the Foundation

This is the most important section for you, since **denoising literally is the mechanism diffusion models are built on** — DDPMs are trained to iteratively remove noise, which is precisely your competition's core task, just repeated many times instead of once.

4.1 DDPM — "Denoising Diffusion Probabilistic Models" (Ho, Jain, Abbeel, NeurIPS 2020)

Problem it solved: Earlier generative models (GANs, VAEs) were either hard to train stably (GANs — mode collapse, adversarial instability) or produced blurry samples (VAEs). DDPM offered a third path with **stable training and high sample quality**.

Architecture, simply explained: - **Forward process:** Start from a real clean image and add a little bit of Gaussian noise, over and over, for many steps (e.g., 1000 steps), until the image is pure noise. - **Reverse process:** Train a **U-Net** to predict, at each step, what noise was added — i.e., learn to slightly denoise the image, one small step at a time. - At generation time, you start from pure random noise and repeatedly apply the trained U-Net denoiser, step by step, until a clean, realistic image emerges.

A diffusion model is a neural network that learns to gradually denoise data starting from pure noise. Note how directly this describes your task — the entire generative mechanism is a denoiser network.

Novelty: Made diffusion models practically competitive for the first time by simplifying the training objective to "just predict the noise that was added" — a much easier and more stable target than earlier diffusion formulations.

Inspired by: Builds on Sohl-Dickstein et al. (2015)'s original diffusion-based generative modeling theory, and is mathematically connected to Song & Ermon (2019)'s score-based generative modeling.

Why it's relevant to you: The U-Net-based denoising network *inside* DDPM is architecturally almost identical to what you need for single-pass denoising. You don't need the full multi-step generative sampling process for your task (that would be far too slow for a "small model, low FLOPs" competition) — but the **denoising U-Net architecture itself**, and the idea of predicting a noise/residual rather than the clean image directly, is a strong, well-validated design choice you can borrow directly.

4.2 DDRM — "Denoising Diffusion Restoration Models" (Kawar et al., NeurIPS 2022)

Problem it solved: Training a *separate* diffusion model for every possible restoration task (denoising, deblurring, super-resolution, inpainting...) is wasteful. *This work introduces Denoising Diffusion Restoration Models (DDRM), an efficient, unsupervised posterior sampling method that takes advantage of a pre-trained denoising diffusion generative model for solving any linear inverse problem.*

Architecture, simply explained: Takes one already-trained, general-purpose DDPM (trained just to generate realistic images, not for any specific restoration task) and, at inference time only, guides its denoising steps using the corrupted input image as a constraint — without retraining the network at all.

Novelty: A single generative prior (the pretrained DDPM) can solve **many different restoration tasks** at inference time just by changing how you constrain the sampling process — no task-specific training needed.

Inspired by / built on: Directly built on top of DDPM (Ho et al., 2020) and reuses publicly released pretrained diffusion checkpoints (e.g., from OpenAI's guided-diffusion repo).

Why it's relevant to you: This paper is strong conceptual support for why a **generative prior helps generalization**: because the model's underlying "knowledge of what a real image looks like" doesn't depend on knowing the specific corruption type in advance, it naturally handles corruption types outside its explicit training set — precisely your zero-shot requirement. However, DDRM as-is requires many sampling steps at inference (expensive, hurts your FLOPs score), so it's more useful as an inspiration for the mechanism than a direct copy.

4.3 SR3 — "Image Super-Resolution via Iterative Refinement" (Saharia et al., 2022)

Problem it solved: Extends DDPM to be **conditional** — instead of generating a random image from noise, it generates a high-resolution image conditioned on a specific low-resolution input.

Architecture, simply explained: A conditional U-Net denoiser that takes both the noisy intermediate image *and* the low-quality input image (concatenated as extra input channels) at every diffusion step, so the network always "knows" what input it's trying to be faithful to while it denoises.

Novelty: Showed that the DDPM training recipe generalizes cleanly to conditional image-to-image tasks (not just unconditional generation), simply by feeding in the conditioning image alongside the noisy one.

Why it's relevant to you: This conditioning trick — concatenating your corrupted input image as an extra channel alongside whatever the network is processing — is directly usable even outside a full diffusion pipeline; it's a simple, well-validated way to keep a generative network "grounded" in the specific input it needs to restore.

4.4 Latent Diffusion Models / Stable Diffusion (Rombach et al., CVPR 2022)

Problem it solved: Running diffusion directly on full-resolution pixel images (like DDPM/SR3 do) is very computationally expensive, since every one of the ~1000 denoising steps runs on the full image resolution.

Architecture, simply explained: First, train a **VAE (Variational Autoencoder)** to compress images into a much smaller latent space (e.g., $256 \times 256 \times 3$ pixels $\rightarrow$ $32 \times 32 \times 4$ latent features). Then, run the entire diffusion process (forward noising + reverse denoising U-Net) in this small latent space instead of pixel space. Finally, decode the denoised latent back into a full-resolution image using the VAE decoder.

Novelty: Dramatically reduces the FLOPs of diffusion models by moving the expensive iterative part into a much smaller space, while a lightweight VAE handles the compression/decompression. This is the architecture underlying Stable Diffusion.

Why it's relevant to you — directly addresses your FLOPs concern: If you want to use a diffusion-style generative approach but are worried about compute cost, working in a **compressed latent space** (even a much simpler one than full Stable Diffusion's) is the standard trick to make diffusion-based restoration affordable. You could denoise directly in pixel space with a single forward pass (cheaper, more like DDRM/SR3's single-step denoiser core) or explore a lightweight VAE + latent-space refinement if you want the "generative" framing with lower cost than full pixel-space diffusion.

4.5 DiffPIR — "Denoising Diffusion Models for Plug-and-Play Image Restoration" (Zhu et al., 2023)

Problem it solved: Classical "Plug-and-Play" restoration methods use a generic denoiser (like BM3D or a simple CNN) as a building block inside an iterative optimization loop to solve restoration problems. DiffPIR asks: what if we plug in a much more powerful **generative diffusion model** as that denoiser instead?

Architecture, simply explained: *Integrates the traditional plug-and-play method into the diffusion sampling framework*, treating each optimization sub-step as a denoising problem solved by a pretrained diffusion model, avoiding the need to backpropagate through the entire diffusion model.

Novelty: Achieves strong restoration quality using at most 100 sampling steps (much less than the ~1000 of vanilla DDPM), by cleverly combining classical optimization structure with a generative denoiser.

Inspired by: Builds on DDPM and DDRM's idea of using a pretrained diffusion model as a generic prior, but restructures the iteration to converge faster.

Why it's relevant to you: A good reference if you want to explore fewer-step generative denoising — the "use a generative denoiser inside a small number of refinement iterations" pattern could inspire a 2-4 step lightweight generative refinement head on top of a cheap backbone, trading a little extra compute for potentially better generalization than a single feed-forward pass.

4.6 GenDeg — "Diffusion-Based Degradation Synthesis for Generalizable All-in-One Image Restoration" (Rajagopalan et al., 2024)

Problem it solved: Notices that all-in-one restoration models are often bottlenecked not by architecture, but by the **limited diversity of their synthetic training corruptions**.

Architecture, simply explained: Uses a diffusion model in the opposite direction from usual — instead of denoising, it's trained to *generate realistic corrupted versions* of a clean image, producing far more diverse and realistic corrupted/clean training pairs than a small fixed list of hand-coded noise functions could.

Novelty: Treats the *training data generation* itself as a generative modeling problem, rather than only using generative models for the restoration step.

Why it's relevant to you: Directly maps onto your competition's Section 3.2, which warns that the held-out set may contain undisclosed degradation types. Even without implementing a full diffusion-based corruption generator, this paper's core lesson is: **broaden and randomize your synthetic corruption pipeline as much as possible**, since narrow synthetic training data is consistently identified as the main generalization bottleneck.

4.7 Defusion / Visual-Instructed Degradation Diffusion (Luo et al., CVPR 2025)

Problem it solved: Unifies restoration across 8 different degradation types in a single diffusion model, guided by "visual instructions" (visual examples showing what kind of damage is present) instead of manual task labels.

Architecture, simply explained: *Visual instructions are constructed from visual grounds to demonstrate the visual effects of the image degradations; visual instructions are tokenized and contrasted with the "clean" (null) visual grounds; finally, the visual instruction tokens guide the denoising diffusion model which estimates the degradation according to the hint of visual instructions.* The core network uses a **U-Net** backbone with **DDIM sampling** for faster inference.

Novelty: Rather than requiring the model to guess the degradation type purely from pixels, it uses a learned "visual instruction" representation as an additional generative conditioning signal — a modern hybrid of the AirNet-style degradation-fingerprint idea and full diffusion generation.

Why it's relevant to you: This is the most "state-of-the-art" example of blind, unified, generative restoration in the timeline — useful as a north star for what a large-scale version of this approach looks like, even though its full multi-task diffusion pipeline is heavier than what a mini-competition submission would need.

5. Efficient / Transformer Backbones Used Inside These Generative Pipelines

Even generative pipelines need an efficient "workhorse" network to do the actual per-step denoising. Two dominant choices recur across nearly every paper above:

5.1 Restormer (Zamir et al., CVPR 2022)

A U-Net-shaped efficient Transformer. Its key trick, **Multi-Dconv Head Transposed Attention (MDTA)**, *performs query-key feature interaction across channels rather than the spatial dimension*, giving it linear rather than quadratic complexity relative to image size. Used as the backbone inside PromptIR, AdaIR, and many diffusion-based restoration U-Nets when Transformer-quality features are needed.

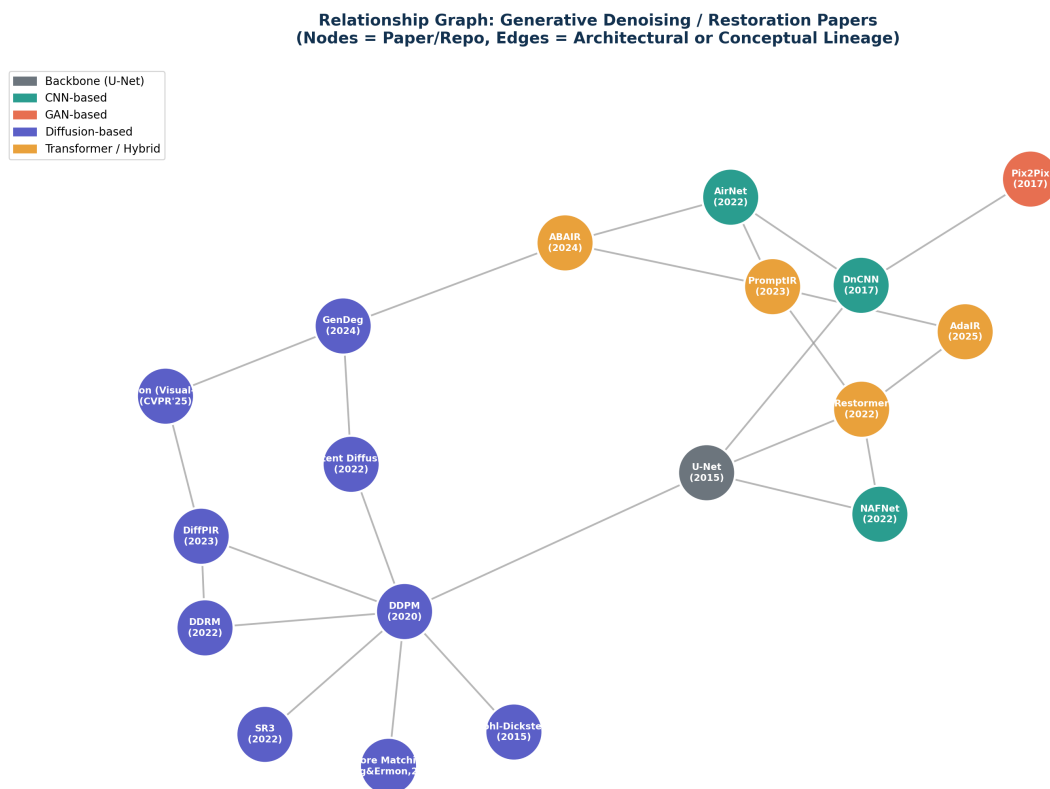
5.2 NAFNet (Chen et al., ECCV 2022)

Removes or replaces nonlinear activation functions with simpler operations like multiplication, simplifying the model structure while maintaining restoration quality across multiple tasks. In direct FLOPs comparisons, NAFNet measures substantially cheaper than Restormer at the same resolution (roughly 505.5 GFLOPs for NAFNet vs. 1128.9 GFLOPs for Restormer at 512×512, with correspondingly lower latency). This is the more FLOPs-friendly choice if compute budget is tight.

Practical implication for you: Whether you go GAN-based (Pix2Pix-style) or Diffusion-based (DDPM/SR3-style), the internal generator/denoiser network is very often a U-Net. You can make that internal U-Net cheap by borrowing NAFNet's design (simple gate, no expensive activations) or make it more expressive with Restormer-style efficient attention if you have FLOPs budget to spare.

6. How the Pieces Connect — Relationship Graph

The diagram below shows how the papers/repos above relate to one another: **nodes** are papers or repositories, **edges** represent a direct architectural or conceptual lineage (e.g., "built on top of," "backbone reused from," "directly inspired by"), and **colors** group papers by their core generative-architecture family.



How to read it: - The **U-Net** node sits centrally because nearly every family (CNN, Diffusion, Transformer) reuses its encoder-decoder-with-skip-connections shape as a backbone. - The **DDPM** node is the hub of the diffusion cluster (blue) — DDRM, SR3, Latent Diffusion, and DiffPIR all build directly on it, and GenDeg /

Defusion extend that lineage further into training-data synthesis and multi-task unified restoration. - The **DnCNN** → **AirNet** → **PromptIR** → **AdaIR** / **ABAIR** chain (green/orange) shows the "blind all-in-one restoration" lineage, evolving from a plain CNN residual learner into contrastive degradation-awareness, then prompt-based conditioning, then frequency-awareness and adapter-based specialization. - **Pix2Pix** sits at the edge, connected to DnCNN — the two both process corrupted → clean pairs, one via adversarial GAN loss, the other via plain residual regression, marking the historical GAN vs. CNN fork before diffusion models took over as the dominant generative approach.

7. Recommended Plan & Alternative Approaches (Short)

Recommended starting plan: 1. Backbone: a small **U-Net**, built NAFNet-style (Simple Gate blocks, no expensive activations) for low FLOPs. 2. Training objective: predict the **clean image directly**, or the **noise/residual** (DDPM-style residual prediction tends to train more stably) — try both, keep whichever validates better. 3. Conditioning: concatenate the corrupted image as input (SR3-style), and optionally add a small AirNet/PromptIR-style lightweight degradation-aware embedding to help the model implicitly recognize "what's wrong" without labels. 4. Training data: broad, randomized, on-the-fly corruption pipeline (GenDeg's lesson) across natural + low-light + medical domains. 5. Keep it **single-pass** (no multi-step diffusion sampling) to protect your FLOPs score — treat this as "one denoising step of a diffusion model," not the full generative sampling loop.

Alternative approaches to try, briefly, if time allows: - **Conditional GAN (Pix2Pix-style):** add a small discriminator and adversarial loss on top of your U-Net to push outputs toward sharper, more realistic textures — likely to help your DISTS (perceptual) score specifically, at some added training complexity (not inference cost). - **Few-step latent refinement:** compress with a small VAE/autoencoder, run 2–4 lightweight denoising refinement steps in the compressed space (DiffPIR/Latent-Diffusion-inspired), then decode — a middle ground between a single feed-forward pass and full multi-step diffusion, if a pure single-pass model underperforms. - **Frequency-aware branch (AdaIR-inspired):** add a simple FFT/wavelet-based branch alongside your main backbone to make the model naturally more robust to degradation types it hasn't seen, since many unseen corruptions still behave predictably in frequency space. - **Pretrained diffusion prior at inference only (DDRM-style):** if you have access to (or can cheaply train) a general-purpose unconditional image diffusion model, you could reuse it purely at inference time as a generative prior — powerful, but likely too FLOPs-expensive for this competition's scoring unless heavily distilled.

8. Full Reference List

1. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI 2015.
2. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015). *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*.
3. Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). *Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising (DnCNN)*. IEEE TIP.
4. Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2017). *Image-to-Image Translation with Conditional Adversarial Networks (Pix2Pix)*. CVPR 2017.

5. Song, Y., & Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. NeurIPS 2019.
6. Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS 2020.
7. Kavar, B., Vaksman, G., & Elad, M. (2022). *Denoising Diffusion Restoration Models*. arXiv:2201.11793.
8. Saharia, C., Ho, J., Chan, W., Salimans, T., Fleet, D. J., & Norouzi, M. (2022). *Image Super-Resolution via Iterative Refinement (SR3)*.
9. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR 2022.
10. Zamir, S. W., Arora, A., Khan, S., Hayat, M., Khan, F. S., & Yang, M.-H. (2022). *Restormer: Efficient Transformer for High-Resolution Image Restoration*. CVPR 2022.
11. Chen, L., Chu, X., Zhang, X., & Sun, J. (2022). *Simple Baselines for Image Restoration (NAFNet)*. ECCV 2022.
12. Li, B., Liu, X., Hu, P., Wu, Z., Lv, J., & Peng, X. (2022). *All-in-One Image Restoration for Unknown Corruption (AirNet)*. CVPR 2022.
13. Zhu, Y., Zhang, K., Liang, J., Cao, J., Wen, B., Timofte, R., & Van Gool, L. (2023). *Denoising Diffusion Models for Plug-and-Play Image Restoration (DiffPIR)*. CVPR Workshops 2023.
14. Potlapalli, V., Zamir, S. W., Khan, S., & Khan, F. S. (2023). *PromptIR: Prompting for All-in-One Blind Image Restoration*. NeurIPS 2023.
15. Cui, Y., Zamir, S. W., Khan, S., Knoll, A., Shah, M., & Khan, F. S. (2024/2025). *AdaIR: Adaptive All-in-One Image Restoration via Frequency Mining and Modulation*. ICLR 2025.
16. Serrano-Lozano, D., Herranz, L., Su, S., & Vazquez-Corral, J. (2024). *Adaptive Blind All-in-One Image Restoration (ABAIR)*.
17. Rajagopalan, S., Nair, N. G., Paranjape, J. N., & Patel, V. M. (2024). *GenDeg: Diffusion-Based Degradation Synthesis for Generalizable All-in-One Image Restoration*.
18. Luo, et al. (2025). *Visual-Instructed Degradation Diffusion for All-in-One Image Restoration (Defusion)*. CVPR 2025.
19. Jiang, J., Zuo, Z., Wu, G., Jiang, K., & Liu, X. (2024). *A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends*. arXiv:2410.15067.

9. Notable GitHub / Hugging Face Repositories

Name	Link	Relevance
<code>bahjat-kawar/ddrm</code>	github.com/bahjat-kawar/ddrm	Official DDRM code — pretrained diffusion models reused for restoration
<code>yuanzhi-zhu/DiffPIR</code>	github.com/yuanzhi-zhu/DiffPIR	Official DiffPIR plug-and-play diffusion restoration code
<code>cszn/DnCNN</code>	github.com/cszn/DnCNN	Classic residual-learning denoiser, good minimal baseline
<code>SUSI-Lab/Awesome-GAN-based-Image-Restoration</code>	github.com/SUSI-Lab/Awesome-GAN-based-Image-Restoration	Curated list of GAN-based restoration papers/code

Name	Link	Relevance
Harbinzzy/All-in-One-Image-Restoration-Survey	github.com/Harbinzzy/All-in-One-Image-Restoration-Survey	Companion repo to the AiOIR survey, tracks the latest papers
Hugging Face huggingface.co/blog/annotated-diffusion	—	Step-by-step annotated PyTorch DDPM implementation, a good place to learn the U-Net + diffusion training loop hands-on
YHL04/ddpm	github.com/YHL04/ddpm	Compact, minimal DDPM reference implementation

This document was compiled from published research papers, official code repositories, and surveys retrieved from arXiv, OpenReview, GitHub, and Hugging Face as of August 2026, for reference purposes for the Mini Competition on generative image denoising (Foundation Models & Generative AI course).